MINI PROJECT REPORT

on

DISTRIBUTED COMMUNICATION NETWORK

Submitted by

ROOPESH O R (20323085)

in partial fulfillment of requirement for the award of the degree of

BACHELOR OF TECHNOLOGY
in
ELECTRONICS AND COMMUNICATION

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI - 682022

APRIL 2026

DIVISION OF ELECTRONICS ENGINEERING
SCHOOL OF ENGINEERING
COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
KOCHI-682022

CERTIFICATE

Certified that the mini project report entitled ***"DISTRIBUTED COMMUNICATION NETWORK"*** *is a bonafide work of* ***ROOPESH O R*** *towards the partial fulfillment for the award of the degree of B.Tech in Electronics and Communication of Cochin University of Science and Technology, Kochi-682022.*

Mini Project Guide **Head of the Division**

Acknowledgment

The satisfaction that accompanies the successful completion of this project would be incomplete without the mention of the people who made it possible, without whose constant guidance and encouragement would have made efforts go in vain. We consider ourself privileged to express gratitude and respect towards all those who guided us through the completion of this project. It is our privilege to express our sincerest regards to our project coordinator, Mrs. Krishnapriya C V, Mrs. Mumtaz P Ali and Mr. Unni A M for their valuable inputs, able guidance, encouragement, whole-hearted cooperation and constructive criticism throughout the duration of our project. We deeply express our sincere thanks to our Head of Department, Dr. Deepa Sankar for encouraging and allowing us to present the project at our department premises. We take this opportunity to thank all our lecturers who directly or indirectly helped our project. We pay our respects and love to our parents and all other family members and friends for their love and encouragement throughout our career. Last but not the least we express our thanks to our friends for their cooperation and support.

Abstract

This project presents the design and implementation of a distributed, low-cost communication system specifically for person localization in a forest using a mesh network. The system is built on STM32 and uses nRF24L01+ transceivers to achieve 2.4GHz wireless links with across the network. Unlike traditional star topologies, the nodes operate within a software-defined mesh, utilizing a controlled flooding algorithm to ensure near fault tolerant message delivery across nodes, even in challenging RF environments. A primary feature of the implementation is the custom path-recording within packets transmitted by node; each data packet carries a telemetry header that logs the unique identifiers of every intermediary relay node. This mechanism allows for tracking of person's location within the network's topology and facilitates fast rescue operation.

Table of contents

List of Figures

List of Tables

List of Abbreviations

ACK:	Acknowledgment
AODV:	Ad-hoc On-demand Distance Vector
COTS:	Commercial off-the-shelf
DIP:	Dual in-line package
ESB:	Enhanced ShockBurst
GNSS:	Global Navigation Satellite Systems
GPIO:	General Purpose I/O
I2C:	Inter-Integrated Circuit
IDE:	Integrated Development Environment
IoT:	Internet of Things
LOS:	Line-of-Sight
M2M:	Machine-to-Machine
MAC:	Media Access Control
MANET:	Mobile Adhoc Network
PA+LNA:	Power Amplifier and Low Noise Amplifier
SPI:	Serial Peripheral Interface
SRAM:	Static RAM
UAV:	Unmanned Aerial Vehicle
USART:	Universal Synchronous/Asynchronous Receiver Transmitter
VANET:	Vehicular Adhoc Network
WSN:	Wireless Sensor Netwok

Chapter 1:
Introduction

Communication infrastructure is the most critical logistical component of any Search and Rescue operation. In deep, dense forests, traditional communication networks such as cellular grids or centralized VHF radio repeaters are either non-existent or rendered ineffective due to severe signal attenuation caused by foliage and uneven terrain. When search teams or lost individuals operate in these disconnected zones, the lack of a reliable communication lifeline drastically reduces the probability of a successful rescue.

To overcome the limitations of centralized networks, decentralized Wireless Sensor Netwok (WSN) offer an alternative. By deploying a localized, ad-hoc mesh network, communication can be maintained by "hopping" signals from one node to another until they reach a central command station. This project demonstrates the design and implementation of such a system: a distributed multi-hop communication network designed specifically to relay critical state changes (such as an SOS signal) across separated nodes.

1.1 System Architecture

The core objective of this prototype is to validate the feasibility of decentralized packet routing using low-cost hardware. The physical demonstration consists of a localized four-node network:

- One ***Transmitter***: Simulating an edge device triggered by a user in distress.
- One ***Receiver***: Acting as the gateway or rescue command center.
- Two ***Relay Nodes***: Acting as repeaters. Because these nodes are battery-powered, forest guards could place these modules in different parts of forest to instantly establish a network.

The computation of each node is handled by an STM32 series microcontroller, which handles interfacing of nRF24L01 RF module. These four nodes are expected to be organized in a manner shown in Figure 1.1.

The system assumes that each node is already geolocated. The Based on the information in the packet the receiving station can determine which node was nearest to the transmitter and hence can obtain a approximate location of the transmitter. A GNSS module can be attached to each node to locate its position on the fly, reducing manual labor of tagging each node. However the addition of a GNSS module can increase the cost of whole system several times. Also once the nodes are placed int various locations, their position is fixed, hence GNSS module will become redundant.

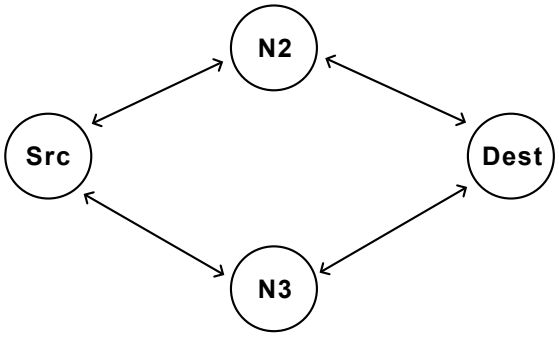


Figure 1.1: Intended organization of nodes

1.2 Protocol Design

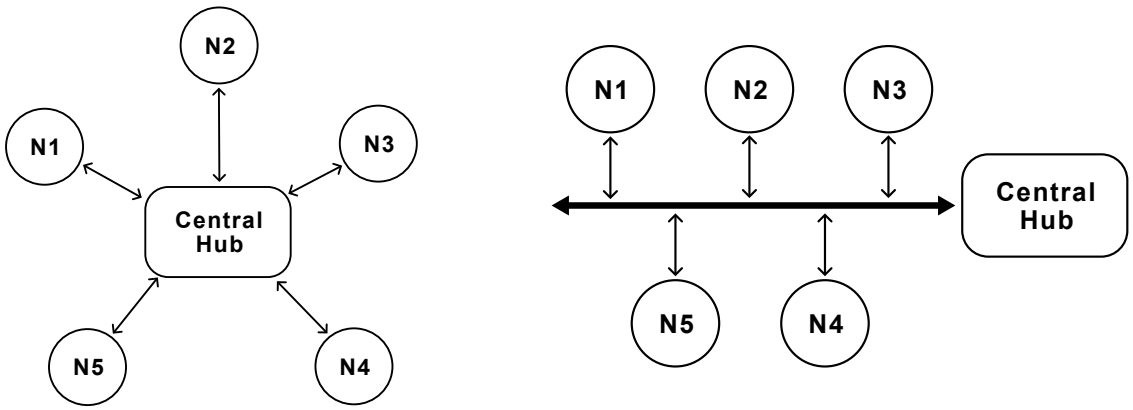
A primary decision in this project was the not to use heavy, conventional mesh routing protocols like Ad-hoc On-demand Distance Vector (AODV) or Zigbee. While these protocols are highly efficient in static, enterprise-grade networks, they require significant effort to get started and to maintain routing tables and calculate shortest-path vectors. Since no GNSS system is incorporated the node distance is not directly computable.

Considering the data available to each node we decided to implemented Controlled Flooding. In a flooding architecture, an intermediate node simply broadcasts any packet it receives that is not addressed to itself. To prevent the network from collapsing under infinite routing loops (known as a "broadcast storm"), the flooding is strictly controlled via software. Each node maintains a localized "seen stack" – an array of details of recently transmitted packets. If a packet's signature exists in the stack, it is dropped; if it is new, it is forwarded. This ensures redundancy and that an SOS signal will aggressively find the fastest physical path through the forest to the receiver, without the nodes needing to map the network topology in advance.

In emergency communications, "fire-and-forget" transmission is unacceptable; the Transmitting node must know if the distress signal successfully reached the command station (Receiver). To signal the successful data delivery, the system implements an application-layer Acknowledgment (ACK) mechanism. When the final destination node successfully receives the targeted packet, it generates a distinct ACK packet and routes it back through the mesh to the original transmitter. The transmitter node after transmitting SOS goes to listening state, and upon receiving this targeted ACK, the node provides physical feedback (via an LED indicator) to confirm that the transmission successfully reached the command station.

Chapter 2:
Literature Review

Historically, network systems have been studied for wide area communication [1] and distributed computing. In earlier days sensor networks relied on centralized, ***star topology*** architectures. In these networks, all edge devices communicate directly with a central gateway. While simple to implement, star topologies possess a single point of failure and are strictly limited by the physical transmission range of the central node . In complex environments – such as industrial facilities with heavy RF attenuation, dense agricultural fields, or robotic swarm deployments – maintaining direct line-of-sight to a central hub is often physically impossible or expensive. A ***common bus topology*** was also considered but it suffers from the same problem – need of central orchestrator.



(a) Star Topology (b) Common Bus Topology

Figure 2.1: Star and Common Bus topologies

In these scenario decentralized ***Mesh Networking*** has emerged as a viable solution [2]. In a mesh topology, individual nodes can act as originators of data and/or active relays (routers) for their peers. This cooperative data forwarding extends the effective range of the network far beyond the capability of single transmitter system and provides much higher fault tolerance; if one node fails, the network can re-routes traffic through other existing paths. These network are self-organizing and does not require a central orchestrator.

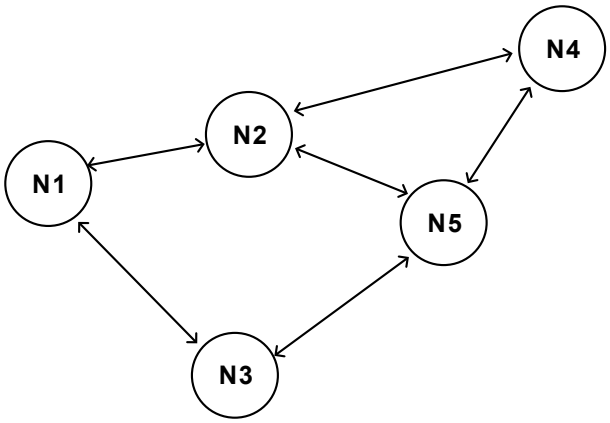


Figure 2.2: Mesh network topology

Despite the advantages of mesh topologies, implementing them in highly constrained environments is a significant engineering challenge. Standard mesh protocols, such as Zigbee has performance and interoperability issues for large networks. Furthermore, many commercial mesh solutions are proprietary or rely on heavy, power-hungry transceivers.

There are more complex systems like Mobile Adhoc Networks (MANETs), and Vehicular Adhoc Networks (VANETs) which are suited for very dynamic environments. We also explored *Meshtastic* a sophisticated off-grid mesh networking LoRa protocol [3]. However, the problem our project tries to tackle involves a relatively static environment where nodes remain in their place for long run. Hence routing scheme such as **Controlled Flooding** [4] best suited for our problem domain. Controlled flooding is easier to implement while being light on resources.

Chapter 3:
Implementation Details

The system is made with portability in mind. The algorithm is custom developed based on already available Controlled flooding algorithms [4], [5]

3.1 Block diagram

The operation of the system is as following:

1. User sends SOS message
2. Transmitter transmits
3. Routing network routs the packet
4. SOS station receives it. (only if it is ment for this receiver)
5. Sends a ACK message through netwrok.
6. On successfull reception, an appropriate LED is turned on.

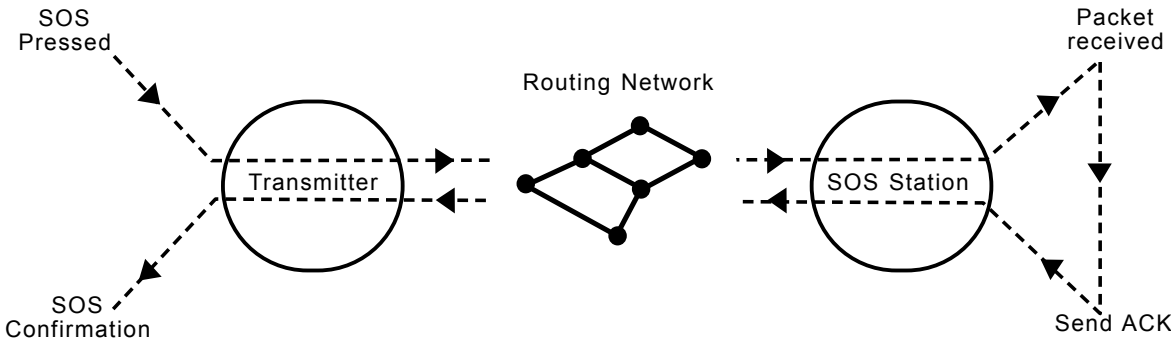


Figure 3.1: Block diagram

3.2 Component Selection

The key components used are STM32F103 (C6T6 variant), and nRF24L01+. Other minor components used are listed in *Section 3.4: Bill of materials*

3.2.1 STM32F103

The STM32F103 is the main processing unit of each node in the network. It is based on the ARM Cortex-M3 architecture and is responsible for controlling all node operations, including packet creation, packet forwarding, routing logic, path recording, and communication with peripheral devices. The microcontroller offers sufficient processing speed, low power consumption, and multiple communication interfaces such as SPI and UART, which make it well suited for embedded wireless applications. In this project, the STM32F103 handles the software-defined mesh logic and

manages the interaction with the nRF24L01+ transceiver. The microcontroller has following key specifications:

- 16 KiB flash memory and 6 KiB SRAM.
- 72 MHz Clock frequency
- Supports 2.0 to 3.6V input
- Has one SPI, one I2C, 2 Universal Synchronous/Asynchronous Receiver Transmitter (USART)
- 37 General Purpose I/O (GPIO)

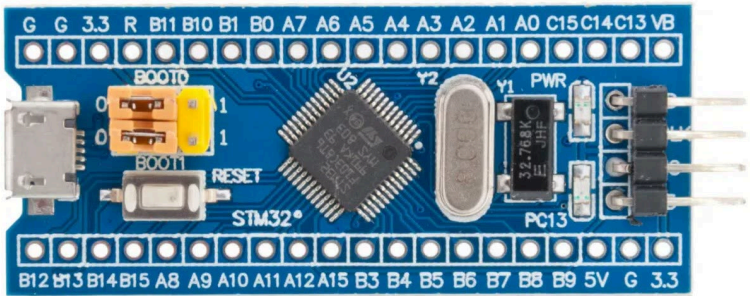


Figure 3.2: STM32F103 development board

There are many STM32 variants (such as F401) in market however we choose this particular model due to the requirements:

- Very low cost
- No need of floating point operations (hence not considering F401)
- Enough number of GPIO pins for future expansion of peripherals.

However programming the board was not a easy task. ST Microelectronics recommends using ***STM32CubeIDE*** – the official IDE for STM boards – for creating and flashing programs onto the development boards. The board we are currently using is unfortunately unsupported by the IDE. Hence the board was programmed using Arduino IDE by installing STM board package [6] and setting appropriate settings (shown in Figure 3.8 in *Section 3.5: Software*) while flashing. The program was hence written in Arduino's C++ language.

3.2.2 nRF24L01+

The nRF24L01+ module is used to establish wireless communication between nodes in the mesh network. It operates in the 2.4 GHz ISM band and supports low-power, short-range digital communication with good data transfer rates. The transceiver communicates with the STM32F103 through the SPI interface and is responsible for transmitting and receiving packets between neighboring nodes. In this project, the nRF24L01+ enables multi-hop communication through controlled flooding, allowing packets to propagate across the network while recording the relay path followed. The module has following specifications: [7]

- Very low standby current of 22µA
- Supports input voltage of 1.9 to 3.6V, (which can be powered by STM board itself).

- 12 mA current consumption during operation.
- Data rate of 250Kbps
- Data transfer through Serial Peripheral Interface (SPI)

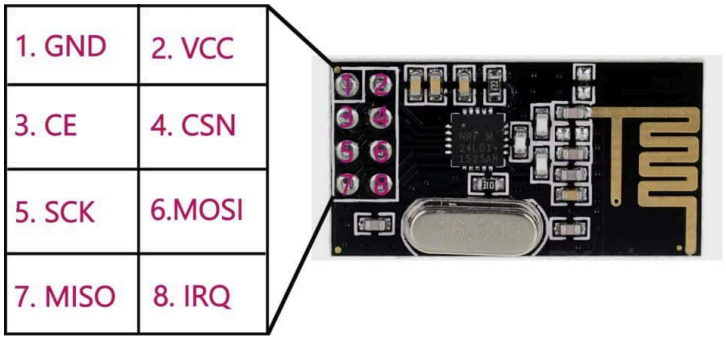


Figure 3.3: nRF24L01+ and its pinout

We considered other alternatives such as nRF24L01+PA+LNA version which has power amplifier for transmission and a low noise amplifier for receiver extending the Line-of-Sight (LOS) communication range to 1km. However the module was five times expensive than the base model significantly rising the budget. Moreover due to its extensive range, it is not suitable for the demonstration of our network as we will have to setup large area for testing. Similarly LoRa modules were also not considered due to its wide range and high abstraction from hardware. FS1000A was another module taken for consideration for being inexpensive. But from our past experiences, the module is often unstable and has higher rate of failure. Moreover these modules have poor manufacturing quality.

The nRF24L01 was chosen considering the drawbacks of other alternatives.

3.3 Hardware

The nRF module is interfaced to STM board via SPI through hardware SPI peripheral of STM32. An indicator LED used to indicate the status of transmission is attached to `PB11` pin of STM. Built-in LED in the development board – which is accessed by `PC13` is used to indicate reception of radio packet reducing the need for additional LED and current limiting resistor. For visual feedback green and blue LEDs (`D1`) where used along with a 1kΩ resistor (`R1`) for current limiting. For initiating the transmission of SOS signal, a push button (`SW1`) is used which is connected to ground via 1kΩ resistor (`R2`). Choice of 1kΩ is well-satisfies maximum I/O source/sink current capability of STM32 which is 25mA [8]

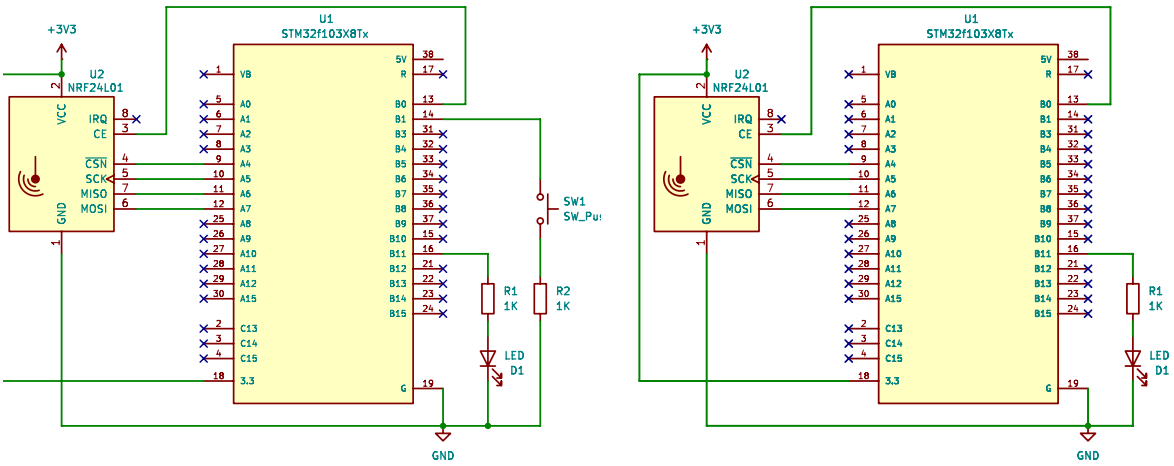


Figure 3.4: Transmitter and Receiver circuit diagrams

While creating finalized prototype, we soldered female berg strip onto prototype board. For transmitter, we were experimenting various configurations hence it was not soldered in a prototype board.

The relay nodes are powered by a 3.7V 18650 Li-ion battery with 2600mAh capacity. The battery has enough capacity to run the nodes continuously for weeks. To use the battery in the nodes, usually a voltage regulator is needed. However the onboard voltage regulator in STM32 development board which has a high drop off voltage of 1V to 1.2V. Hence applying battery directly to 5V pin of board will result in very reduced output voltage. For demonstration purpose, we decided to use partially charged battery with voltage close to 3.4 to 3.5V and connect it directly to 3.3V. The absolute voltage ratings are still satisfied as the maximum voltage for both STM32 IC and nRF module is 3.6V.

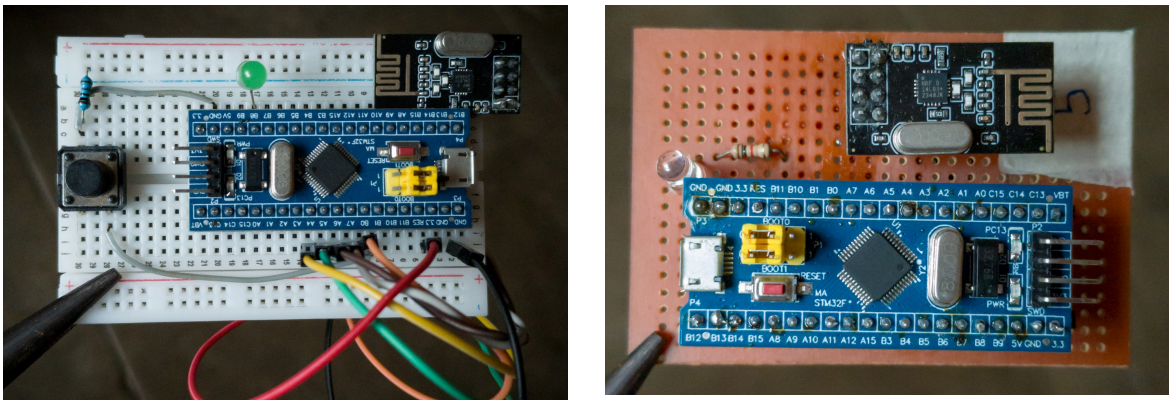


Figure 3.5: Transmitter (left) and receiver(right) modules

3.4 Bill of materials

Although a breadboard was used for a single node project, it was solely ment for testing purpose.

Item	Unit Cost	Units	Total
STM32F103	90	4	360
nRF24L01+	60	4	240
LED	2	4	8
Resistors	1	4	4
Prototype board	60	1	60
Push button	5	1	5
18650 battery	60	2	120
Breadboard 400 points	35	1	35
Total			**₹832**

Table 3.1: Bill of materials

3.5 Software

This section outlines software part of the project including the custom algorithm for controlled flooding.

As discussed previously in *Section 3.2.1: STM32F103*, the program for STM had to be written in Arduino C++ language and was compiled and uploaded via ST-link/V2 [6]. The wiring scheme is shown in Figure 3.7 which was taken from [9]. However other variant like ST-link V2 (Figure 3.6) can be used and works in similar way.

Figure 3.6: ST Link V2

While setting up the Arduino IDE for development few flash settings had to be adjusted as shown in Figure 3.8 The USART support should be set to “Enabled” and USB support option should be set to “CDC” in order for the device to be listed as a serial device and hence can be communicated via programs like Minicom

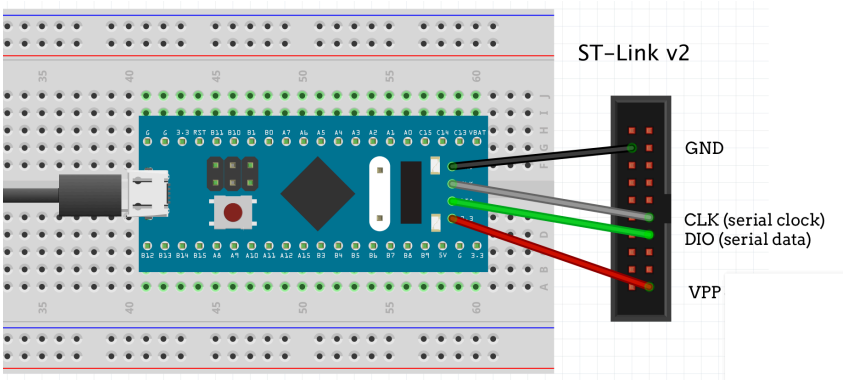


Figure 3.7: Connecting STM32 with ST-Link/V2.

Each nodes has its own node ID. During the prototyping stage these IDs are hard coded in the program. Hence while flashing the program, this ID was changed for each node. However this can be simplified by adding a Dual in-line package (DIP) switch to each node,and setting the address on the fly.

The details of program and algorithm are discussed in following sections.

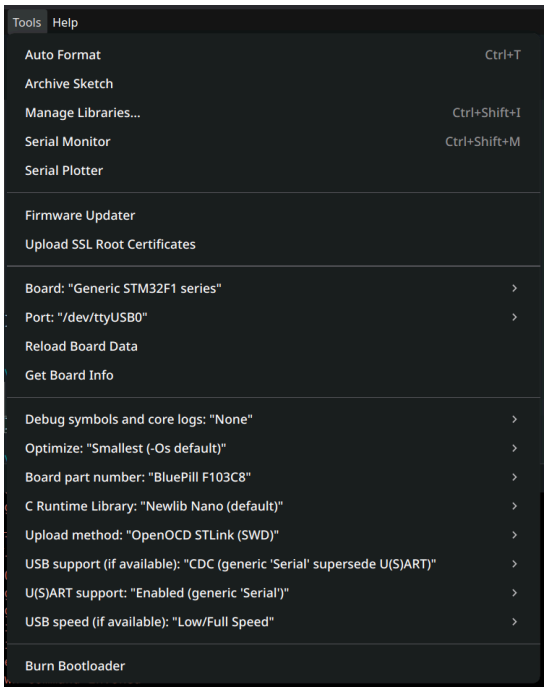


Figure 3.8: Settings to apply while flashing the program to STM32

3.5.1 Transmitter

The transmitter node is initialized by configuring the button and LED GPIO pins. The LED is set to indicate an idle state, and the radio module is checked for proper operation. If the radio is not

responding, the system enters an infinite loop to prevent faulty execution. Once verified, the radio is set to listening mode, preparing it for communication.

Algorithm 3.1: Transmitter Initialization

```
Init Button and LED GPIO pins
LED_SEND ← HIGH
if Radio not responding then
    (Infinite loop)
end
Set radio in listening mode
```

When the SOS button is pressed and the system is not waiting for an acknowledgment, a new transmission is initiated. The LED is turned on to indicate active transmission, and a packet is constructed in memory. A flag (`waitingForACK`) is set to indicate that the node is now expecting an acknowledgment preventing duplicate transmissions. The current time is stored (`lastSOS_Sent`) to track when the packet was sent. Additionally, the message ID (`msgID`) is incremented to uniquely identify each transmission. In the program a short delay of 500ms is added to prevent rapid repeated transmissions which can overload the network.

Algorithm 3.2: SOS Event and Transmission Start

```
if SOS BTN pressed and not waiting For ACK then
    LED_SEND ← LOW
    Rebuild pkt memory buffer
    Transmit pkt
    waitingForACK ← 1
    lastSOS_Sent ← currentTime()
    msgID ← msgID + 1
    Delay 500ms
end
```

When the node is awaiting acknowledgment and it receives a packet, it is checked for validity, duplication, or relevance. If the packet has already been seen or is not an SOS signal, it is discarded. If the packet reaches its intended destination, the system blinks an LED to indicate reception and stores its signature to prevent reprocessing. Expired packets are also discarded to maintain efficiency.

Algorithm 3.3: ACK Reception

```
if waitingForACK and payload received then
    pkt ← Received payload
    if pkt.msg_type != SOS or pkt already seen then
        break
    end
    if pkt destination = This Node then
        Blink Indications
```

```
        Save packet signature
        break
    end
    if pkt expired then
        Save packet signature
        break
    end
end
```

The transmitter can stay in listening mode while waiting for acknowledge signal only for 3 seconds. If no acknowledge is received the system goes in normal state. This delay also prevents the spamming of network with repeated button presses.

3.5.2 Receiver and Repeater

The receiver and repeater nodes shares the same program and logic. Similar to transmitter node, the receiver node is initialized by configuring the required LED GPIO pins, used for indicating reception and transmission activity. The radio module is then initialized and checked for proper operation; if it fails, the system halts to avoid unreliable communication. Once verified, the radio is configured with the appropriate address and settings for receiving data. Finally, the node is set to listening mode, allowing it to continuously monitor for incoming packets.

Algorithm 3.4: Receiver Initialization

```
Init LED GPIO pins
LED_SEND ← HIGH
if Radio not responding then
    (Infinite loop)
end
Set radio in listening mode
```

When a payload is received, the packet is first read and checked against previously seen packets to avoid duplication and unnecessary retransmission. If the packet has already been processed, it is immediately discarded. The algorithm also checks whether the packet has expired (based on hop limit – `ttl` field of packet); expired packets are ignored to prevent network congestion. After validating the packet, the node appends its own ID to the packet path, allowing tracking of the route taken. If the current node is the destination, it indicates successful reception by blinking the RX LED, stores the packet signature to avoid future duplicates, and constructs an acknowledgment (ACK) packet addressed to the original sender. Regardless of whether it is the destination or an intermediate node, the packet's signature is saved to prevent reprocessing. Finally, the packet is transmitted further in the network, and the TX LED blinks to indicate forwarding activity.

Algorithm 3.5: Handle arrival of new package

```
1:  if Payload Received then
2:      pkt ← Receive Packet
3:      if pkt already seen then
4:          break
5:      end
6:      if pkt expired then
7:          Save packet signature
8:          break
9:      end
10:     Append This node's ID to pkt.path
11:     if pkt destination = This Node then
12:         Blink RX LED
13:         Save packet signature
14:         Construct ACK packet using pkt buffer with dest = original sender
15:     end
16:     Save pkt's packet signature
17:     Transmit pkt
18:     Blink TX LED
19: end
```

3.5.3 Packet structure

nRF24L01 Accepts a 32 byte sized memory chunk for transmission. The packet structure in the current protocol implementation is as following:

```
packet = {
    byte msgID;
    byte msg_type;
    byte ttl;
    byte src;
    byte dest;
    byte path[27];
}
```

Description of each fields are following:

1. **`msgID`**: a byte long ID used to identify the message sent by a node. By design 256 unique messages are possible. Every node keeps a internal variable to keep track of the number of messages sent by it, which after each transmission increments it.
2. **`msg_type`**: indicates whether packet is a SOS (value = 0) or ACK (value = 1)
3. **`ttl`**: Time to Live. Specifies the maximum number of hops allowed for a packet.
4. **`src`**: Node ID of the originating node.
5. **`dest`**; Node ID of the final destination node.
6. **`path`**: A stack that stores all the nodes through which the packet was routed. When a node receives a packet and decides to forward it, the node appends its node ID to this field. Hence at the destination node, the receiver can know which node was nearest to the transmitter. If all relay nodes are geolocated, then the location of transmitter can be estimated from this field.

Chapter 4:
Results and Limitations

4.1 Results

Initially direct connection between transmitter and receiver was tested, which was successful. Then we moved on to 3 node and finally to 4 node communication as shown in Figure 4.1. Here the radio modules showed very poor range, hence for transmission to occur, they had to be placed very close to each other. Here transmitter (shown on left) could transmit only up to the nearest repeater which relays it to receiver node (shown on right). Since ACK mechanism was implemented the signal has to travels back to transmitter again through the repeater.

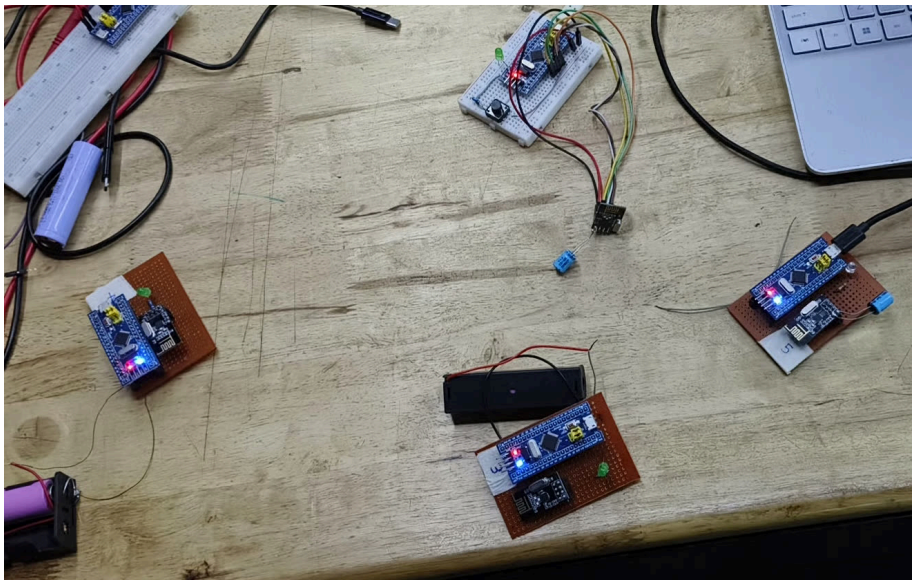


Figure 4.1: Communication with 4 nodes

The output from all nodes were logged through Serial port and are displayed in the console using ***Minicom*** [10], as shown in Figure 4.2.

Later on we tested with all 4 nodes and formed the network which we originally intended to. During operation, on most case the packet travels through both repeaters and reaches the destination at almost the same time. However the radio library we used picks up only one of these packets and the program logs its signature. Hence when the other packet reaches the node, it simply discards it. In similar way, repeater nodes might obtain packets from other repeaters. But the nodes are programmed to detect and discard these duplicate packets, preventing the network from collapsing due to looping.

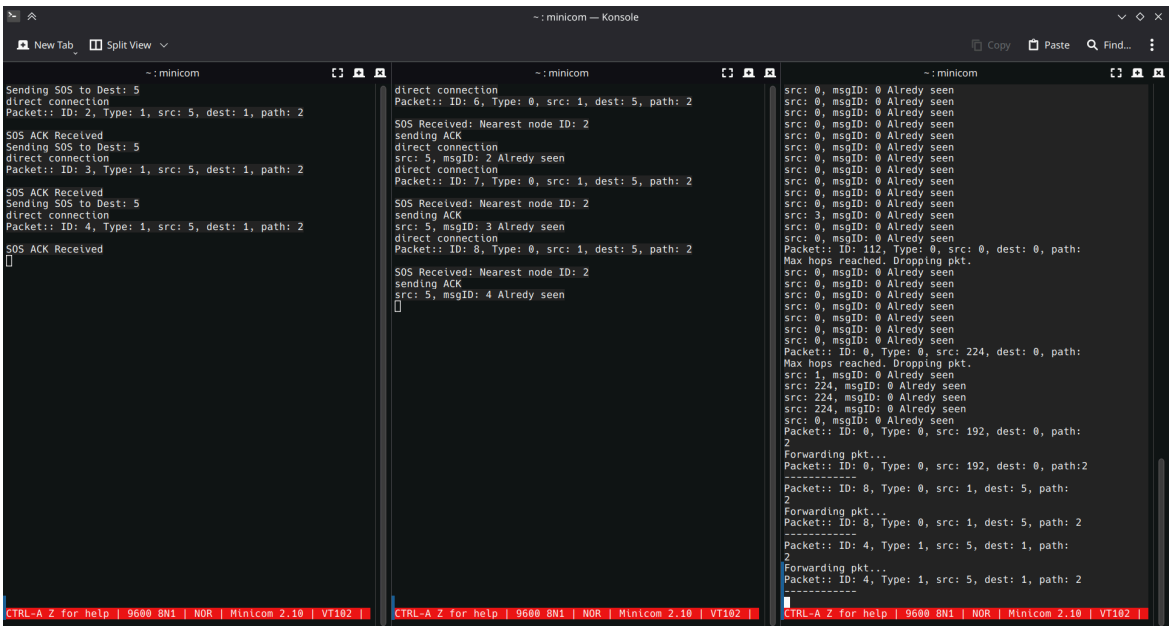


Figure 4.2: Serial output from Transmitter, Receiver and Repeater nodes (left to right)

During the testing we found there was some packet, possibly due to the way the RF module itself works. The details are mentioned in *Section 4.3: limitations*

4.2 Latency Measure

Many serial print function calls and delays were added in the program for debugging purpose. To measure the transmission latency these were removed during the transmission and reception period and only essential logging functionality was left out. The latency here refers to the time period between transmission of SOS signal and reception of a ACK signal. The multiple SOS signals were sent in succession and the latency was logged.

In a direct communication, the latency was highly predictable with an averages value of 1304μs (latency in each transmission is shown in Table 4.1). Since a packet has maximum size of 32 bytes, the maximum date rate for our network scheme is approximately 24 KiB/s

Latency (μs)	1301	1306	1312	1304	1304	1304	1305	1304	1304
Average	**1304μs**								

Table 4.1: Latency for direct communication

```
SOS ACK Received
1301
Sending SOS to Dest: 5

SOS ACK Received
1306
Sending SOS to Dest: 5

SOS ACK Received
1312
Sending SOS to Dest: 5

SOS ACK Received
1304
Sending SOS to Dest: 5
```

Figure 4.3: Serial messages in generated in round trip transmission

For a 3 node communication however, we saw significant increase in latency of almost 30 fold increase. Our speculation for this steep increase is signal attenuation (hence packet loss) as well as Enhanced ShockBurst (ESB) feature of nRF24L01+ chip. Which automatically transmits same packet multiple times in a network

Based on this, the maximum data rate of the communication possible is less than 1KiB/s

Latency(μs)	32198	32210	32023	32034
Average	**32116 μs**			

Table 4.2: Latency for 3 node communication

4.3 limitations

The prototype is not free of limitations, and there are lot of areas that can be improved on. They are listed below.

4.3.1 RF module limitations

The physical RF layer of this prototype is powered by the standard nRF24L01+ transceiver (without the external Power Amplifier and Low Noise Amplifier (PA+LNA)). While these choice keeps us to demonstrate the proof-of-concept of algorithm, it introduces specific limitations:

1. **Range Constraints**: Without a PA+LNA, the transmission range of the standard nRF24L01+ is strictly limited (typically 10 to 30 meters depending on obstacles). In a real forest deployment,

RF absorption by trees would further reduce this range, necessitating a higher density of relay nodes.

2. **MAC-Layer Abstraction**: The nRF24L01+ chip features a built-in hardware protocol called Enhanced ShockBurst (ESB). ESB automatically handles hardware-level ACKs and auto-retransmissions (automatically resending a failed packet up to 3 times, or up to 15 times depending on register settings). While highly useful for simple point-to-point links, this hardware abstraction removes granular software control from the developer. It makes it difficult to implement custom algorithms at the Media Access Control (MAC) layer, as the hardware dictates the timing of the retransmissions independent of software defined mesh logic in STM.

4.3.2 Geolocating Nodes

The task of identifying physical location of nodes is left to the maintainers or personal handling the initial setup. This will not be much of an issue if the network being built is relatively small with few number of nodes. However it can get very complicated as number of nodes increases in very vast networks. Possible solution is to develop a software that can automatically log the ID of each node and the current location of the device. This can be used during setting up of the network.

4.3.3 Security

The prototype has some security flaws which a bad actor can utilize to take down the whole network. Most importantly, the network is susceptible to overflooding due to repeated transmission of SOS signals. This could be solved by modification to existing protocol or using more advanced systems like Meshtastic.

Chapter 5:
Future Scope

1. **Increasing Bandwidth / High-Volume Data:**
   The current system is limited by the 32-byte payload constraint of the nRF24L01+ module. This can be improved using techniques such as packet fragmentation, where large data is split into smaller packets and reassembled at the destination. Pipelining and efficient encoding methods can further enhance throughput. Alternatively an entirely different radio hardware can be incorporated instead which has higher payload capacity. These improvements would enable transmission of more complex data such as sensor outputs beyond simple control signals.

2. **Ad-hoc Networking:**
   The system can be extended into a full Ad-hoc network where nodes dynamically organize without centralized control. By implementing routing protocols like Ad-hoc On-demand Distance Vector (AODV), the network can determine optimal paths automatically which is useful for high data bandwidth domains. This would allow the system to adapt to node failures and topology changes. Such enhancements improve scalability and robustness.

3. **Vehicular Adhoc Networks (VANETs):**
   The project can be extended to VANETs where vehicles communicate in real time. Multi-hop communication enables messages such as accident alerts or traffic updates to propagate across vehicles. Additional features like GNSS integration and low-latency communication would enhance performance. This can contribute to safer and more efficient transportation systems.

4. **Wireless Sensor Netwoks (WSNs):**
   The system is well-suited for Wireless Sensor Network applications in remote environments. Sensor nodes can collect data such as temperature or smoke levels and transmit it across multiple hops. This is useful in applications like wildfire detection and environmental monitoring. Energy-efficient communication strategies can further improve long-term deployment.

5. **Internet of Things (IoT) and Smart Systems:**
   The system can be applied into IoT applications for smart environments. Devices can communicate over a decentralized mesh network for tasks like smart agriculture or home automation. Data collected from multiple nodes can be used for monitoring and decision-making. Future integration with cloud platforms can enable remote access and analytics.

References

[1] Lawrence G. Roberts and Barry D. Wessler, “Computer network development to achieve resource sharing.”

[2] Zhanserik Nurlan and et al., “Wireless Sensor Network as a Mesh: Vision and Challenges.”

[3] Stephen Cass, “Build Long-Range IoT Applications Fast With Meshtastic.” [Online]. Available: https://spectrum.ieee.org/build-iot-apps-with-meshtastic

[4] A. Rahman, W. Olesinski, and P. Gburzynski, “Controlled Flooding in Wireless Ad-hoc Networks.” [Online]. Available: https://web.archive.org/web/20170210205207/http://www.senserf.com/pg/PAPERS/tarp1.pdf

[5] Fatemeh Nourazar and Masoud Sabaei, “An efficient flooding algorithm for Mobile Ad-hoc networks.”

[6] B.Montanari, “How to program and debug the STM32 using the Arduino IDE.” [Online]. Available: https://community.st.com/t5/stm32-mcus/how-to-program-and-debug-the-stm32-using-the-arduino-ide/ta-p/608514

[7] Nordic Semiconductor, “nRF24L01 Single Chip 2.4GHz Transceiver.” [Online]. Available: https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf

[8] ST Microelectronics, “Low-density performance line, ARM-based 32-bit MCU with 16 or 32 KB Flash, USB, CAN, 6 timers, 2 ADCs, 6 com. interfaces.” [Online]. Available: https://www.st.com/resource/en/datasheet/stm32f103c6.pdf

[9] Barnaby Walters, “Programming a Blue/Black Pill STM32 Board with a ST-Link v2.” [Online]. Available: https://waterpigs.co.uk/articles/black-blue-pill-stm32-st-link-connection/

[10] “minicom(1) - Linux man page.” [Online]. Available: https://linux.die.net/man/1/minicom

Appendices

A.1 Transmitter Program

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#define MAX_PATH_SIZE 27
// node ID is never 0
#define DEST_NODE_ID 5
#define THIS_NODE_ID 1
#define MAX_PACKET_SIZE 32
#define MAX_SEEN_STACK 20
#define BTN_SOS_PIN PB1
#define LED_SOS_SEND PC13
#define LED_SOS_ACK PB8
typedef struct __attribute__((packed)) {
  byte msgID;
  byte msg_type; // 0 - sos, 1 - ack
  byte ttl;
  byte src;
  byte dest;
  byte path[MAX_PATH_SIZE];
s } Packet;
// Radio Pins: CE = PB0, CSN = PA4
RF24 radio(PB0, PA4);
const uint64_t address = 0xF0F0F0F0E1LL;
byte seenTxrStack[MAX_SEEN_STACK] = { 0 };
byte seenMsgIDStack[MAX_SEEN_STACK] = { 0 };
int seenStackTop = -1;
byte msgID = 1;
Packet pkt;
byte waitingForACK = 0;
long lastSOS_Sent;
void setup() {
  Serial.begin(9600);
  pinMode(BTN_SOS_PIN, INPUT_PULLUP);
  pinMode(LED_SOS_SEND, OUTPUT);
  pinMode(LED_SOS_ACK, OUTPUT);
  digitalWrite(LED_SOS_SEND, HIGH);
  if (!radio.begin()) {
    Serial.println("Radio hardware not responding!");
    while (1) ;
  }
  radio.openWritingPipe(address);
  radio.openReadingPipe(0, address);
  radio.setPALevel(RF24_PA_MIN);
  radio.startListening();
  Serial.println("Transmitter ready");
}
void loop() {
  if (digitalRead(BTN_SOS_PIN) == LOW && waitingForACK == 0) {
    digitalWrite(LED_SOS_SEND, LOW);
    // Build pkt
    memset(&pkt, 0, sizeof(Packet));
    pkt.msgID = msgID;
    pkt.msg_type = 0;
    pkt.ttl = 10;              // TTL
    pkt.src = THIS_NODE_ID;    // Src node ID
    pkt.dest = DEST_NODE_ID;   // destination node ID
    Serial.print("Sending SOS to Dest: ");
    Serial.println(DEST_NODE_ID);
    transmit(&pkt, sizeof(Packet));
```

```
    waitingForACK = 1;
    lastSOS_Sent = millis();
    msgID++;
    delay(500);
  } else if (waitingForACK == 1 && radio.available()) {
    memset(&pkt, 0, sizeof(Packet));
    radio.read(&pkt, sizeof(Packet));
    if (pkt.msg_type != 1) return;
    if (pkt.path[0] == 0 && pkt.dest == THIS_NODE_ID) {
      Serial.println("direct connection");
      // return;
    }
    for (int j = 0; j < MAX_SEEN_STACK; j++) {
      if (seenTxrStack[j] == pkt.src && seenMsgIDStack[j] == pkt.msgID) {
        // ignore
        Serial.print("src: ");
        Serial.print(pkt.src);
        Serial.print(", msgID: ");
        Serial.print(pkt.msgID);
        Serial.println(" Alredy seen");
        return;
      }
    }
    int i = pathStackTop(pkt);  // no of nodes in path.
    // Destination reached
    if (pkt.dest == THIS_NODE_ID) {
      Serial.println("\nSOS ACK Received");
      digitalWrite(LED_SOS_ACK, HIGH);
      waitingForACK = 0;
      digitalWrite(LED_SOS_SEND, HIGH);
      delay(1000);
      digitalWrite(LED_SOS_ACK, LOW);

      markAsSeen(pkt);
      return ;
    }
    if (pkt.ttl <= 0) {
      markAsSeen(pkt);
      return;
    }
  } else if (waitingForACK == 1 && millis() - lastSOS_Sent > 3000) {
    waitingForACK = 0;
    digitalWrite(LED_SOS_SEND, HIGH);
  }
}
void transmit(const void* data, uint8_t len) {
  radio.stopListening();
  radio.write(data, len);
  radio.startListening();
}
int pathStackTop(Packet pkt) {
  char printBuff[50];
  snprintf(printBuff, sizeof(printBuff),
           "Packet:: ID: %d, Type: %d, src: %d, dest: %d, path: ",
           pkt.msgID, pkt.msg_type, pkt.src, pkt.dest);
  Serial.print(printBuff);
  int i = -1;
  while (i < MAX_PATH_SIZE) {
    i++;
    if (pkt.path[i] == 0) break;
    Serial.print(pkt.path[i]);
  }
  Serial.println();
  return i;
}
```

```
void markAsSeen(Packet pkt) {
  seenStackTop = (seenStackTop + 1) % MAX_SEEN_STACK;
  seenTxrStack[seenStackTop] = pkt.src;
  seenMsgIDStack[seenStackTop] = pkt.msgID;
}
```

A.2 Receiver Program

```
#include <SPI.h>
#include <nRF24L01.h>
#include <RF24.h>
#define MAX_PATH_SIZE 27
#define DEBUG true
#define THIS_NODE_ID 5
#define MAX_PACKET_SIZE 32
#define MAX_SEEN_STACK 20
#define LED_SEND PC13
#define LED_RX PB11
typedef struct __attribute__((packed)) {
  byte msgID;
  byte msg_type;
  byte ttl;
  byte src;
  byte dest;
  byte path[MAX_PATH_SIZE];
} Packet;
// Radio Pins: CE = PB0, CSN = PA4
RF24 radio(PB0, PA4);
const uint64_t address = 0xF0F0F0F0E1LL;
Packet pkt;
byte seenTxrStack[MAX_SEEN_STACK] = { 0 };
byte seenMsgIDStack[MAX_SEEN_STACK] = { 0 };
int seenStackTop = -1;
byte msgID = 1;
void setup() {
  Serial.begin(9600);
  pinMode(LED_RX, OUTPUT);
  pinMode(LED_SEND, OUTPUT);
  if (!radio.begin()) {
    while (1) {
      Serial.println("Radio hardware not responding!");
      delay(100);
    }
  }
  radio.openReadingPipe(0, address);
  radio.openWritingPipe(address);
  radio.setPALevel(RF24_PA_MIN);
  radio.startListening();
  Serial.print("Node started. ID = ");
  Serial.println(THIS_NODE_ID);
}
void loop() {
  if (radio.available()) {
    memset(&pkt, 0, sizeof(Packet));
    radio.read(&pkt, sizeof(Packet));
    for (int j = 0; j < MAX_SEEN_STACK; j++) {
      if (pkt.path[j] == THIS_NODE_ID) {
        // ignore
        Serial.print("src: ");
        Serial.print(pkt.src);
        Serial.print(", msgID: ");
        Serial.print(pkt.msgID);
        Serial.println(" Alredy seen");
        return;
      }
    }
```

```
    int i = pathStackTop(pkt);  // no of nodes in path.
    // Destination reached
    if (pkt.dest == THIS_NODE_ID) {
      Serial.print("\nSOS Received: Nearest node ID: ");
      Serial.println(pkt.path[0]);
      digitalWrite(LED_RX, HIGH);
      delay(1000);
      digitalWrite(LED_RX, LOW);
      markAsSeen(pkt);
      Serial.println("sending ACK");
      Packet pk2;
      memset(&pk2, 0, sizeof(Packet));
      pk2.msgID = msgID;
      msgID++;
      pk2.msg_type = 1;  // ACK
      pk2.ttl = 10;
      pk2.src = THIS_NODE_ID;
      pk2.dest = pkt.src;
      markAsSeen(pk2);
      transmit(&pk2, sizeof(Packet));
      return;
    }
    if (pkt.ttl <= 0) {
      markAsSeen(pkt);
      Serial.println("Max hops reached. Dropping pkt.");
      return;
    }
    // Append node ID
    pkt.path[i] = THIS_NODE_ID;
    pkt.ttl = pkt.ttl - 1;
    Serial.println(pkt.path[0]);
    Serial.println("Forwarding pkt...");
    pathStackTop(pkt);
    transmit(&pkt, sizeof(Packet));
    Serial.println("------------");
  }
}
int pathStackTop(Packet pkt) {
  char printBuff[50];
  snprintf(printBuff, sizeof(printBuff),
           "Packet:: ID: %d, Type: %d, src: %d, dest: %d, path: ",
           pkt.msgID, pkt.msg_type, pkt.src, pkt.dest);
  Serial.print(printBuff);
  int i = -1;
  while (i < MAX_PATH_SIZE) {
    i++;
    if (pkt.path[i] == 0) break;
    Serial.print(pkt.path[i]);
  }
  Serial.println();
  return i;
}
void markAsSeen(Packet pkt) {
  seenStackTop = (seenStackTop + 1) % MAX_SEEN_STACK;
  seenTxrStack[seenStackTop] = pkt.src;
  seenMsgIDStack[seenStackTop] = pkt.msgID;
}
void transmit(const void* data, uint8_t len) {
  radio.stopListening();
  digitalWrite(LED_SEND, LOW);
  radio.write(data, len);
  radio.startListening();
  delay(200);
  digitalWrite(LED_SEND, HIGH);
}
```